



Instituto Politécnico Nacional
Escuela Superior de Cómputo
Ingeniería en
Sistemas Computacionales



Unidad de Aprendizaje:

Inteligencia Artificial

Grupo: 6CV2

(DFS y BFS)

Integrantes:

Barros Martínez Luis Enrique
Bautista Ríos Alfredo
Garcia Martinez Angel Yahel

Profesor: Marco Antonio Castillo

Introducción

La presente práctica aborda nuestro acercamiento a los algoritmos de búsqueda no informados (DFS y BFS), estos consisten en la selección e implementación de estrategias de búsqueda de un estado solución a partir de un estado inicial sin introducir al algoritmo de solución conocimiento sobre el impacto de las transiciones en la exploración del espacio de estados.

Tras ser implementados dichos algoritmos se obtendrá su análisis respecto al tiempo que les toma llegar a el dato en búsqueda con respecto a los casos ingresados.

Dicho de la siguiente manera los programas elevaorados tiene que tener graficos, ademas de estar implementado bajo un grid que simula un laberito

Desarrollo

Se realizará la programación del algoritmo de Búsqueda en anchura (BFS) y Búsqueda en profundidad (DFS) en esta ocasión en lenguaje C++

BFS

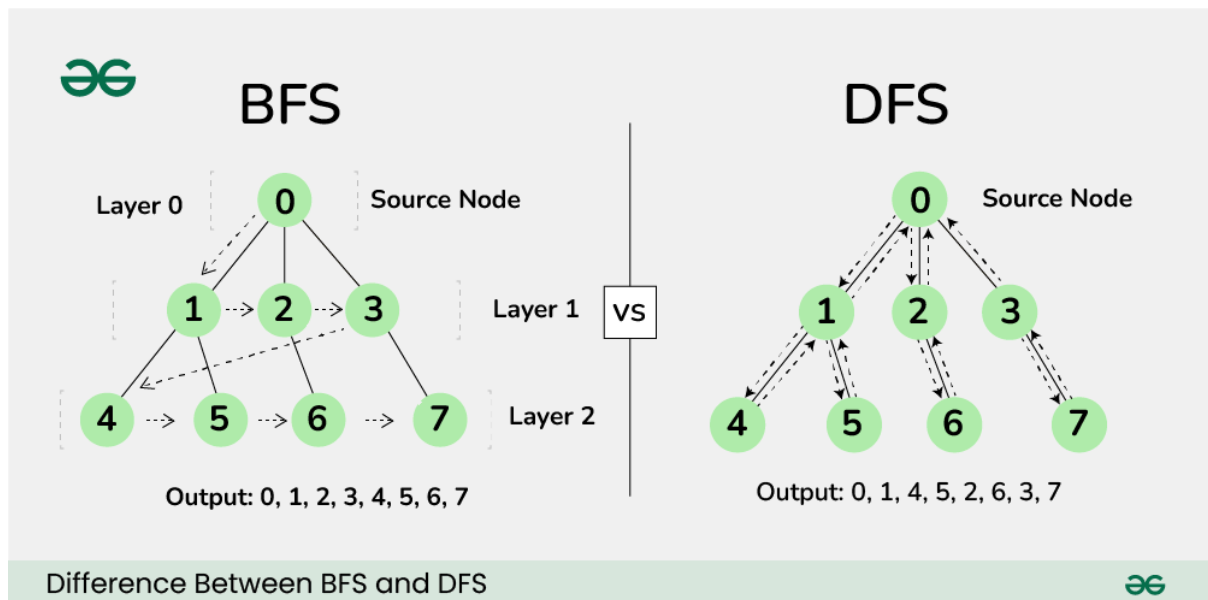
La búsqueda en anchura provee un algoritmo simple (basado en el recorrido de árboles por niveles) para encontrar un estado de solución desde un estado inicial. Al implementarse mediante el recorrido por niveles, se requiere una estructura cola (FIFO, first-in, first-out).

En inteligencia artificial, el algoritmo Breadth-First Search (BFS) es una herramienta esencial para explorar y navegar por diversos espacios de problemas. Al recorrer sistemáticamente estructuras de grafos o árboles, BFS resuelve tareas como la búsqueda de rutas, el enrutamiento de redes y la resolución de acertijos. Este artículo analiza los conceptos básicos de BFS, sus algoritmos y aplicaciones prácticas en IA.

DFS

Una búsqueda en profundidad (DFS) es un algoritmo de búsqueda para lo cual recorre los nodos de un grafo. Su funcionamiento consiste en ir expandiendo cada uno de los nodos que va localizando, de forma recurrente (desde el nodo padre hacia el nodo hijo). Cuando ya no quedan más nodos que visitar en dicho camino, regresa al nodo predecesor, de modo que repite el mismo proceso con cada uno de los vecinos del nodo. Cabe resaltar que si se encuentra el nodo antes de recorrer todos los nodos, concluye la búsqueda.

La búsqueda en profundidad surge como un algoritmo crucial en inteligencia artificial que ofrece tanto conocimientos teóricos como aplicaciones prácticas. Los DFS son una herramienta fundamental para atravesar espacios de estados complejos. A medida que la tecnología evoluciona, DFS sigue siendo un algoritmo fundamental para impulsar avances y ampliar los límites de la investigación y las aplicaciones de la IA.



Código:

El código mostrado a continuación es capaz de recibir cualquier grafo luego de indicarle (número de nodos, número de aristas, y las conexiones), se genera un número aleatorio dentro del rango del número de nodos, el cual será el nodo a encontrar realizando búsquedas con los algoritmos DFS y BFS, primero mostrar el proceso que lleva a cabo la bfs con un cierto delay, terminando este proceso mostrara el camino por medio de una DFS que no siempre tiene que ser igual

```
#include <SFML/Graphics.hpp>
#include <queue>
#include <vector>
#include <iostream>
#include <utility>
#include <cmath>
#include <climits>
using namespace std;
// Laberinto
vector<vector<int>> maze = {
    {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0},
    {0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1},
    {0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0},
    {1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 1, 1, 1},
    {1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 0, 0},
    {0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1},
    {1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 0, 1},
    {0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 1, 0, 0}
};

const int CELL_SIZE = 40; // Tamaño de cada celda en píxeles

//DFS
vector<sf::Vector2i> directions = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
void
dfs(vector<int>current,vector<vector<int>>&visited,vector<vector<sf::Vector2
i>>&prev,int &n,int &m){
    if(visited[current[0]][current[1]])return;
    visited[current[0]][current[1]]=1;
    for(auto dir:directions){
        int nx = current[0] + dir.x; // Nueva posición en x
        int ny = current[1] + dir.y; // Nueva posición en y
        if (nx >= 0 && ny >= 0 && nx < n && ny < m && maze[nx][ny] == 0){
            if(!visited[nx][ny]){
                prev[nx][ny]={current[0],current[1]};
                dfs({nx,ny},visited,prev,n,m);
            }
        }
    }
}
```

```

}
vector<sf::Vector2i> dfs_aux(sf::Vector2i start, sf::Vector2i end) {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Matriz de visitados
    vector<vector<int>> visited(rows, vector<int>(cols,0));
    // Matriz para guardar el nodo previo en el camino más corto
    vector<vector<sf::Vector2i>> prev(rows, vector<sf::Vector2i>(cols, {-1,
-1})));
    // Ejecutar dfs
    dfs({start.x,start.y},visited,prev,rows,cols);
    // Reconstrucción del camino desde el nodo final al inicio
    vector<sf::Vector2i> path;
    for (sf::Vector2i at = end; at != sf::Vector2i{-1, -1}; at =
prev[at.x][at.y]) {
        path.push_back(at);
    }
    reverse(path.begin(), path.end()); // Se invierte el camino para ir de
inicio a fin
    return path;
}
// Algoritmo BFS
vector<sf::Vector2i> bfs(sf::Vector2i start, sf::Vector2i end) {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Matriz de visitados
    vector<vector<int>> visited(rows, vector<int>(cols,0));
    // Matriz para guardar el nodo previo en el camino más corto
    vector<vector<sf::Vector2i>> prev(rows, vector<sf::Vector2i>(cols, {-1,
-1})));
    // Cola para explorar los nodos
    queue<vector<int>>q;
    // Inicializar el nodo de inicio
    q.push({start.x, start.y});
    // Direcciones posibles para moverse (derecha, abajo, izquierda, arriba)
    vector<sf::Vector2i> directions = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
    prev[start.x][start.y]={-1,-1};
    visited[start.x][start.y]=1;
    while(q.size()){
        auto current=q.front();
        q.pop();
        for(auto dir:directions){
            int nx = current[0] + dir.x; // Nueva posición en x
            int ny = current[1] + dir.y; // Nueva posición en y
            if (nx >= 0 && ny >= 0 && nx < rows && ny < cols && maze[nx][ny]
== 0){

```

```

        if(!visited[nx][ny]){
            q.push({nx,ny});
            visited[nx][ny]=1;
            prev[nx][ny]={current[0],current[1]};
        }
    }
}

// Reconstrucción del camino desde el nodo final al inicio
vector<sf::Vector2i> path;
for (sf::Vector2i at = end; at != sf::Vector2i{-1, -1}; at =
prev[at.x][at.y]) {
    path.push_back(at);
}
cout<<endl;
reverse(path.begin(), path.end()); // Se invierte el camino para ir de
inicio a fin
return path;
}

int main() {
    int rows = maze.size(); // Número de filas
    int cols = maze[0].size(); // Número de columnas

    // Crear ventana
    sf::RenderWindow window(sf::VideoMode(cols * CELL_SIZE, rows *
CELL_SIZE), "Dijkstra Maze Solver");

    sf::Vector2i start{-1, -1}, end{-1, -1}; // Coordenadas del inicio y fin
    int selecting = 0; // Bandera 0 inicio 1 fin

    // Bucle principal de la aplicación
    while (window.isOpen()) {
        sf::Event event;
        while (window.pollEvent(event)) {
            if (event.type == sf::Event::Closed)
                window.close(); // Cerrar ventana

            // Detección de clics del mouse para inicio y fin
            if (event.type == sf::Event::MouseButtonPressed) {
                int x = event.mouseButton.y / CELL_SIZE; // Coordenada x en
celdas
                int y = event.mouseButton.x / CELL_SIZE; // Coordenada y en
celdas

                if (x >= 0 && y >= 0 && x < rows && y < cols && maze[x][y] ==
0) {
                    if (selecting==0) {

```

```

        start = {x, y}; // Asignar inicio
        end={-1,-1};
        selecting^=1; // Pasar a seleccionar el fin
    } else {
        end = {x, y}; // Asignar fin
        selecting^=1; // Pasar a seleccionar inicio, Si
continua
    }
}
}

window.clear(sf::Color::White); // Limpiar la ventana

// Dibujar el laberinto
for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        sf::RectangleShape cell(sf::Vector2f(CELL_SIZE - 1, CELL_SIZE
- 1));

        cell.setPosition(j * CELL_SIZE, i * CELL_SIZE);

        // Color de las celdas
        if (maze[i][j] == 1)
            cell.setFillColor(sf::Color::Black); // Muros o barreras
        else
            cell.setFillColor(sf::Color::White); // Espacio
transitable

        // Colorear inicio y fin
        if (start == sf::Vector2i(i, j))
            cell.setFillColor(sf::Color::Green); // Inicio
        if (end == sf::Vector2i(i, j))
            cell.setFillColor(sf::Color::Red); // Fin

        // Dibujar borde negro a cada celda
        cell.setOutlineThickness(1);
        cell.setOutlineColor(sf::Color::Black);

        window.draw(cell); // Dibujar celda
    }
}

// Dibujar el camino más corto si el inicio y fin están definidos
if (start != sf::Vector2i{-1, -1} && end != sf::Vector2i{-1, -1}) {
    auto path = bfs(start, end);
    window.display();
    sf::sleep(sf::seconds(2));
    for (auto &pos : path) {

```

```

        sf::ConvexShape triangulo;
        triangulo.setPointCount(3);
        triangulo.setPoint(0, sf::Vector2f(pos.y * CELL_SIZE, pos.x *
CELL_SIZE)); // Vértice superior
        triangulo.setPoint(1, sf::Vector2f(pos.y *
CELL_SIZE+CELL_SIZE-1, pos.x * CELL_SIZE)); // Vértice inferior izquierdo
        triangulo.setPoint(2, sf::Vector2f(pos.y * CELL_SIZE, pos.x *
CELL_SIZE+CELL_SIZE-1)); // Vértice inferior derecho
        triangulo.setFillColor(sf::Color::Cyan); // Color del camino
        triangulo.setOutlineThickness(1);
        triangulo.setOutlineColor(sf::Color::Black);
        window.draw(triangulo); // Dibujar celda del camino
        window.display();
        sf::sleep(sf::seconds(0.5));
    }
    path=dfs_aux(start,end);
    for (auto &pos : path) {
        sf::ConvexShape triangulo;
        triangulo.setPointCount(3);
        triangulo.setPoint(0, sf::Vector2f(pos.y * CELL_SIZE, pos.x *
CELL_SIZE+CELL_SIZE-1)); // Vértice superior
        triangulo.setPoint(1, sf::Vector2f(pos.y *
CELL_SIZE+CELL_SIZE-1, pos.x * CELL_SIZE+CELL_SIZE-1)); // Vértice inferior
izquierdo
        triangulo.setPoint(2, sf::Vector2f(pos.y *
CELL_SIZE+CELL_SIZE-1, pos.x * CELL_SIZE)); // Vértice inferior derecho
        sf::RectangleShape cell(sf::Vector2f(CELL_SIZE - 1, CELL_SIZE
- 1));

        triangulo.setFillColor(sf::Color::Yellow); // Color del
camino

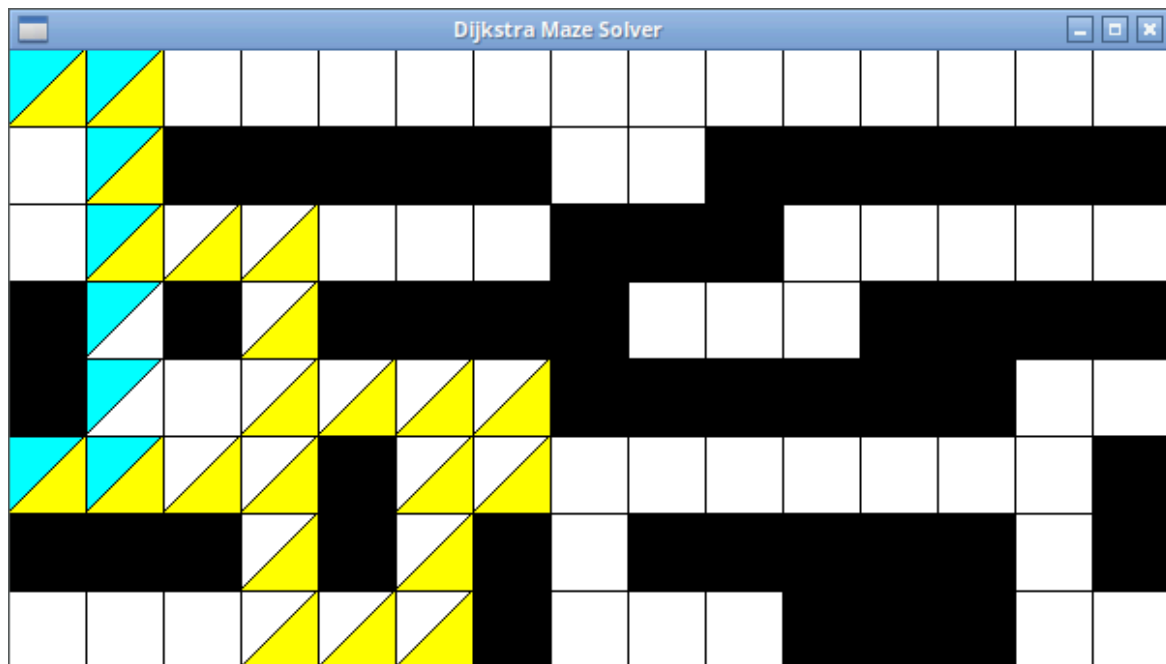
        triangulo.setOutlineThickness(1);
        triangulo.setOutlineColor(sf::Color::Black);
        window.draw(triangulo); // Dibujar celda del camino
        window.display();
        sf::sleep(sf::seconds(0.5));
    }
    window.display();
    sf::sleep(sf::seconds(2));
}

window.display(); // Mostrar la ventana
}

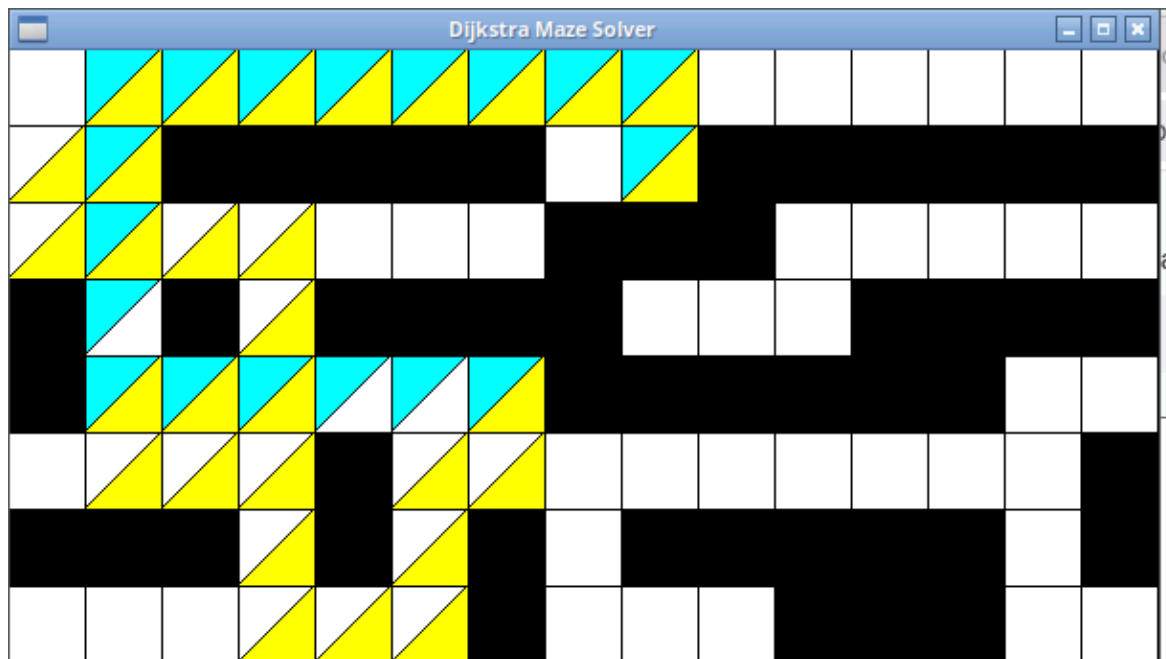
return 0;
}

```

Ejemplo Caso 1:



Ejemplo Caso 2:



Conclusión

La conclusion mas importante de la practica es que la bfs nos ayuda a encontrar un nodo con el menor numero de pasos , razon por la cual la bfs se utiliza en algoritmos de caminos cortos como el dijkstra, pero eso no hace menos importante a los algoritmos como la BFS ya que es un algoritmo mas rapido de implementar que una bfs, pero tiene una desventaja la pila recursiva, para un numero mayor de datos esto podria ser un problema pero no demasiado grande, con lo cual siempre podemos recurrir a un BFS. En conlcusion se puedenm usar ambos para casi la mayoria de datos, unos tienen ciertas propiedades y otros son mas sencillos de implementar.

